

2. sed (Stream Editor)

Role: The Replacer.

sed is used for basic text transformation. It reads input and modifies it based on commands you provide, most commonly for "find and replace."

- **Syntax:** sed [options] 'command' [file]

Practical Examples

- **Basic Search and Replace:**

Replace the first occurrence of "old" with "new" in every line.

```
sed 's/old/new/' file.txt
```

- **Global Search and Replace:**

Replace *all* occurrences of "old" with "new".

```
sed 's/old/new/g' file.txt
```

- **Delete Specific Lines:**

Delete the 5th line of a file.

```
sed '5d' data.txt
```

- **In-place Editing:**

Save the changes directly to the file (use with caution!).

```
sed -i 's/127.0.0.1/localhost/g' config.conf
```

3. awk (Pattern Scanning and Processing)

Role: The Reporter.

awk is a complete programming language designed for processing columns of data. It treats a file like a table (rows and columns).

- **Syntax:** `awk 'pattern { action }' [file]`

Practical Examples

- **Print Specific Columns:**

Print the 1st and 3rd columns of a file (space-separated by default).

```
awk '{ print $1, $3 }' data.txt
```

- **Using Custom Delimiters:**

Print usernames (1st column) from the `/etc/passwd` file, which uses `:` as a separator.

```
awk -F ":" '{ print $1 }' /etc/passwd
```

- **Conditional Processing:**

Print lines where the 3rd column value is greater than 500.

```
awk '$3 > 500' storage.txt
```

- **Summing Values:**

Add up all numbers in the 2nd column and print the total.

```
awk '{ sum += $2 } END { print sum }' sales.txt
```

The Workflow: Combining Tools with Pipes (|)

The true power of Linux comes from using these tools together. A common workflow is to find data with `grep`, clean it with `sed`, and format it with `awk`.

Example: Find all unique IP addresses in an access log that encountered a "404" error.

Bash

```
cat access.log | grep "404" | awk '{print $1}' | sort | uniq
```

Quick Comparison Table

Tool	Focus	Best For
grep	Lines	Finding specific patterns or keywords.
sed	Characters/ Words	Find-and-replace or line deletion.
awk	Columns/Fields	Data extraction, math, and report generation.

•

Looking at your screenshot, you have some successful sed commands and some that gave errors. Let's break down how sed works and why those specific errors happened.

sed stands for **Stream Editor**. Its most common job is to search for a word and replace it with another, but it can also delete lines.

1. The Substitute Command (s)

You successfully ran: `sed 's/PRICE/AMOUNT/g' inventory.txt`

- `s`: This tells sed to **Substitute**.
 - `/PRICE/`: This is what you are looking for (the "Find").
 - `/AMOUNT/`: This is what you want to change it to (the "Replace").
 - `g`: Stands for **Global**. This ensures every "PRICE" on the line is changed, not just the first one.
-

2. Fixing Your Errors

In the screenshot, you tried `sed 'ID' inventory.txt` and got an error.

- **The Problem:** sed always needs a command letter (like s for substitute or d for delete). Just typing 'ID' doesn't tell sed what to *do* with it.
- **The Fix (to delete a line containing ID):**

Bash

```
sed '/ID/d' inventory.txt
```

- **The Fix (to find and replace ID with something else):**

Bash

```
sed 's/ID/IDENTIFIER/' inventory.txt
```

You also tried sed 'ld' inventory.txt.

- **The Problem:** You likely meant 1d (the number one + d).
- **The Fix (to delete the first line):**

Bash

```
sed '1d' inventory.txt
```

Note: The number 1 refers to the line number, and d means delete.

3. Making Changes Permanent (-i)

You used sed -i 's/1200/900/' inventory.txt.

- **-i (in-place):** This is the most important flag. Without it, sed just prints the change to your screen. With it, sed actually **saves** the change into the inventory.txt file.
-

Summary Table for sed

Action	Command Example	What it does
Replace	sed 's/old/new/' file	Changes "old" to "new" on screen.
Save Replace	sed -i 's/old/new/' file	Permanently saves the change.

Action	Command Example	What it does
Delete Line 1	<code>sed '1d' file</code>	Removes the header row.
Delete by word	<code>sed '/Notebook/d' file</code>	Removes any line containing "Notebook".

- **The Fix (to delete a line containing ID):**

`sed '/ID/d' inventory.txt`

- **The Fix (to find and replace ID with something else):**

`sed 's/ID/IDENTIFIER/' inventory.txt`

You also tried `sed 'ld' inventory.txt`.

- **The Problem:** You likely meant `ld` (the number one + d).
- **The Fix (to delete the first line):**

Bash

`sed 'ld' inventory.txt`

```

student@student: ~
102:IPHONE:800:8
103:NOTEBOOK:300:3
EOF
student@student:~$ sed 'ld' inventory.txt | awk -F: '{ $3 > 500 { print $2 " is expensive" } }'
> MACBOOK is expensive
>
> ^C
student@student:~$ sed 'ld' inventory.txt | awk -F: '{ $3 > 500 { print $2 " is expensive" } }'
MACBOOK is expensive
>
> ^C
student@student:~$ cat << EOF > inventory.txt
ID:PRODUCT:PRICE:STOCK
101:MACBOOK:1200:10
102:IPHONE:800:8
103:NOTEBOOK:300:3
EOF
student@student:~$ cat << EOF > inventory.txt
> _
> ^C
student@student:~$ cat inventory.txt
ID:PRODUCT:PRICE:STOCK
101:MACBOOK:1200:10
102:IPHONE:800:8
103:NOTEBOOK:300:3
student@student:~$ sed 's/MACBOOK/TYPEBOOK/g' inventory.txt
ID:PRODUCT:PRICE:STOCK
101:TYPEBOOK:1200:10
102:IPHONE:800:8
103:NOTEBOOK:300:3
student@student:~$ awk '/BOOK/ {print}' inventory.txt
101:MACBOOK:1200:10
103:NOTEBOOK:300:3
student@student:~$

```